

Project Phoenix

CY015 — Attack Testing Plan

Cybersecurity Department | Hardhat Enterprises | Deakin University

Field	Details
Sub Team Leader	Himanshu Khanna
Team Members	Himanshu, Rasanjana, MD Isa, Inderjeet
Date	May 2026
Deadline	May 2026

1. What is This Document?

This document is the attack testing plan for CY015. It covers everything the team needs to run practical attack tests against the Phoenix API. This includes what endpoints are being tested, which attack tactics are being used, who is responsible for what, what tools are needed and how results should be recorded.

The following endpoints were already completed by Naren Reddy Bhoomidi and are not part of this plan:

- Data Ingestion
- Cyber Threat
- Risk Assessment
- Dashboard Activity
- Geolocations

2. API Reference

2.1 Endpoint Groups

API Group	Endpoints	What It Controls
Authentication	POST /auth/register, /auth/login, /auth/refresh, /auth/logout	Who can get into Phoenix and how sessions work
Dashboard	GET /dashboard/overview, /dashboard/charts, /dashboard/activity	What data users can see on the main dashboard
Hazards	GET /hazards, /hazards/:hazardId	Active disaster event data from BoM and CFA

Cyber Threats	GET /threats, /threats/:threatId	Cyber threat intelligence records
Risk Assessments	GET /risk-assessments, /risk-assessments/:assessmentId	Linked hazard-threat correlation scores
Geolocation	GET /locations, POST /data-ingestion/locations	Geographic data for disasters and threats
Data Ingestion	POST /api/ingestion/stream	The pipeline that feeds all data into Phoenix
Metadata	GET /meta/event-statuses, /meta/seasons, /meta/reference-days	Reference data for dropdowns and filters

2.2 RBAC Permissions Matrix

Role	Sign In	View Dashboard	View Data	Create Data	Manage Users
System Admin	Yes	Yes	Yes	No	Yes
Analyst	Yes	Yes	Yes	No	No
End User	Yes	Limited	Limited	No	No
Ingestion Service	No	Internal	Internal	Yes	No

3. Who is Doing What

3.1 Himanshu — Sub Team Leader

Himanshu is responsible for the most critical endpoints — authentication — as well as coordinating the team, reviewing all results and compiling the final vulnerability record.

Endpoint Group	Attack Tactics	Tools
POST /auth/login	Brute force and credential stuffing	Python, Burp Suite
POST /auth/login	SQL injection on login fields	SQLMap, Burp Suite
POST /auth/refresh + /auth/logout	Token replay after logout	Postman, Burp Suite
POST /auth/register	Privilege escalation via role field	Postman
Multiple endpoints	RBAC access control testing	Postman

3.2 Rasanjana

Rasanjana is responsible for testing the threat intelligence, risk assessment and dashboard endpoints. His work builds directly on the STRIDE analysis he completed in CY005.

Endpoint Group	Attack Tactics	Tools
GET /threats, /threats/:threatId	SQL injection, RBAC testing, unauthenticated access	Postman, Burp Suite, SQLMap
GET /risk-assessments, /risk-assessments/:assessmentId	Parameter tampering, RBAC testing, IDOR testing	Postman, Burp Suite
GET /dashboard/overview, /charts, /activity	Unauthenticated access, expired token, RBAC, parameter tampering	Postman, Burp Suite

3.3 MD Isa

MD Isa is responsible for testing the data ingestion stream and hazard endpoints — both high priority areas that require careful testing.

Endpoint Group	Attack Tactics	Tools
POST /api/ingestion/stream	False data injection, malformed payload, unauthenticated access, oversized payload	Postman, Burp Suite, Python
GET /hazards, /hazards/:hazardId	SQL injection, IDOR testing	Postman, Burp Suite, SQLMap

3.4 Inderjeet

Inderjeet runs the OWASP ZAP automated scan across all endpoints before anyone starts manual testing. He also handles geolocation and metadata endpoint testing.

Task	Details	Tools
OWASP ZAP Automated Scan	First action once test environment is confirmed. Share baseline report with full team before manual testing begins.	OWASP ZAP
GET /locations, POST /data-ingestion/locations	SQL injection, false location data injection	Postman, SQLMap
GET /meta/event-statuses, /meta/seasons, /meta/reference-days	Unauthorised access testing	Postman

3.5 Quick Summary

Team Member	Responsible For	Priority
Himanshu	Authentication (5 endpoints), coordination, vulnerability recording (Row 88)	Critical
Rasanjana	Threat endpoints (2), Risk Assessment endpoints (2), Dashboard endpoints (3)	High
MD Isa	Data Ingestion Stream (4 attacks), Hazard endpoints (2)	Critical to High
Inderjeet	OWASP ZAP scan, Geolocation (2), Metadata (3)	High to Lower

4. Attack Tactics

Tactic 1 → Brute Force and Credential Stuffing Basically we just keep sending wrong passwords to the login endpoint over and over using a Python script to see if the system blocks us at some point. If it never blocks us that is a serious problem.

Target: POST /api/users/auth/login | Tool: Python, Burp Suite

Tactic 2 → Token Replay After Logout We login, grab the refresh token, then logout. After that we try using that same token again. If it still works after logout that means stolen tokens dont get properly killed which is bad.

Targets: POST /auth/login, POST /auth/logout, POST /auth/refresh | Tool: Postman, Burp Suite

Tactic 3 → Privilege Escalation via Role Field We register a new account and just put role: admin in the request and see if the system accepts it. If we login and it says admin then literally anyone can make themselves an admin.

Target: POST /api/users/auth/register | Tool: Postman

Tactic 4 → RBAC Access Control Testing We login as an End User and try to access things only Analysts or Admins should see. The system should give us a 403 Forbidden. If it doesnt that is a fail.

Targets: GET /threats/:threatId, dashboard functions | Tool: Postman

Tactic 5 → SQL Injection We put SQL injection payloads into the request fields to see if the system properly cleans the input. If it doesn't the database could leak data it shouldn't or even run our injected commands.

Targets: GET /hazards, GET /threats, GET /risk-assessments | Tool: SQLMap, Burp Suite

Tactic 6 → False Data Injection via Stream Endpoint We try to POST fake data into the stream using an Analyst token which shouldn't have write access there. If it goes through that is a critical problem.

Target: POST /api/ingestion/stream | Tool: Postman

Tactic 7 → Malformed Payload Injection We send broken or oversized data to see how the API handles it. A good API rejects it cleanly without giving away any internal info in the error message.

Targets: POST /api/ingestion/stream, POST /auth/login, POST /auth/register |

Tool: Burp Suite, Python

Tactic 8 → Parameter Tampering We intercept a normal request using Burp Suite and change the parameters before they hit the server. The server should catch this and reject it no matter what the client sends.

Targets: GET /hazards/:hazardId, GET /risk-assessments/:assessmentId | **Tool:** Burp Suite

5. Test Execution Order

Order	What to Test	Why
1	Authentication endpoints	If login security is broken, everything else is exposed
2	Data ingestion stream endpoint	Most dangerous if compromised — false data could enter the whole Phoenix pipeline
3	Hazard and threat endpoints	Contains the most sensitive real-time intelligence data
4	Dashboard endpoints	Main user-facing interface — needs proper access control
5	Risk assessment endpoints	Drives alert decisions — needs protection from tampering
6	Geolocation and metadata endpoints	Lower risk — quick checks at the end

6. Tools Required

Tool	What It Does	Who Uses It	Download
Postman	Sends API requests to Phoenix endpoints	Everyone	postman.com/downloads
Burp Suite Community	Intercepts and modifies API requests	Himanshu, Rasanjana, MD Isa	portswigger.net
Python 3	Runs automated scripts for brute force testing	Himanshu, MD Isa	python.org/downloads
SQLMap	Automated SQL injection testing	Himanshu, Rasanjana, MD Isa	sqlmap.org

OWASP ZAP	Automated API scan for vulnerabilities	Inderjeet	zaproxy.org/download
-----------	--	-----------	----------------------

7. Postman Setup

Setting up Postman before testing:

1. Open Postman and on the left side make a new Collection. Give it the name Phoenix CY015 Attack Tests
2. Now make a new Environment from the left side bottom and name it Phoenix Test Environment
3. Inside that environment add these words one by one in the Variable column — base_url, access_token, refresh_token, admin_token, analyst_token, end_user_token. Do not fill in any values yet, just add the names for now
4. Now make a new Request inside the collection. Change GET to POST and type this in the URL box — {{base_url}}/api/users/auth/login. Then click on Body, select raw and change Text to JSON. In the box that appears type the username and password fields
5. When you successfully login you will get a response. Copy the access_token value from that response and paste it into the access_token variable in your environment. From now on add this to the Headers of every request you send — Key: Authorization, Value: Bearer {{access_token}}

8. How to Record Test Results

Every test must be documented using the format below. A test without a screenshot and written result is not considered complete.

Field	What to Write
Test ID	A unique reference such as AT-01, AT-02. Keep numbering consistent across the team.
Endpoint Tested	The exact API endpoint URL with the full path
Attack Tactic	Which tactic from Section 5 was used
Tool Used	Postman, Burp Suite, Python, SQLMap or OWASP ZAP
What We Did	Plain description of exactly what was done step by step
Expected Result	What should have happened if the security control is working
Actual Result	What actually happened when the test was run
Evidence	Screenshot filename — every test needs at least one screenshot
Status	Pass, Partial Pass or Fail
Recommendation	If the test failed, say specifically what needs to be fixed

Status	What It Means	What to Do
Pass	The security control worked as expected	Document and move to the next test
Partial Pass	The control worked but has a gap	Document the gap and write a recommendation
Fail	The control is not working — the attack got through	Tell Himanshu straight away — do not wait

9. Rules for Testing

- Only test on the confirmed isolated test environment — never on the live production system
- Only test the endpoints listed in this plan — do not go outside agreed scope
- Take a screenshot of every single response — no exceptions
- If a Critical vulnerability is found, tell Himanshu immediately — do not wait until all tests are done
- Do not attempt to access real user data or real victim data at any point
- Do not start testing until the backend team has confirmed the test environment is ready

10. Environment Requirements

The following must be confirmed before any live testing begins:

Requirement	Status
Test environment URL from backend team	Pending
Test user accounts for all 4 roles (Admin, Analyst, End User, Data Ingestion)	Pending
Written permission from team leader to run security tests	Received
Postman installed and workspace set up	Done
Burp Suite Community installed	Done
Python 3 installed	Done
SQLMap installed	Done
OWASP ZAP installed	Done

11. Further Work Summary

Team Member	Further Work	Deadline	Status
Himanshu	Run 5 authentication tests, review all team results, compile final vulnerability record (Row 88)	May 2026	Pending
Rasanjana	Run threat, risk assessment and dashboard endpoint tests, document results, send to Himanshu	May 2026	Pending
MD Isa	Run data ingestion stream and hazard endpoint tests, document results, send to Himanshu	May 2026	Pending
Inderjeet	Run OWASP ZAP scan first and share report, run geolocation and metadata tests, document results, send to Himanshu	May 2026	Pending